МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Методичні вказівки до лабораторної роботи № 3

з дисципліни «Вебтехнології та розробка вебзастосувань»

на тему: «Фронтенд фреймворки»
для студентів першого (бакалаврського) рівня вищої освіти спеціальності F3 Комп'ютерні науки освітньо-професійної програми "Комп'ютерні науки (Обчислювальний інтелект смарт-систем)"

Затверджено
на засіданні кафедри
Автоматизованих Систем Управління.
Протокол № 1-2025/2026 « 2 » вересня 2025р.

Львів - 2025

Методичні вказівки до виконання лабораторної роботи №3 «Фронтенд фреймворки» з дисципліни «Вебтехнології та розробка вебзастосувань» для студентів першого (бакалаврського) рівня вищої освіти спеціальності F3 Комп'ютерні науки освітньо-професійної програми "Комп'ютерні науки (Обчислювальний інтелект смарт-систем)" / Укл. Артем Казарян – Львів: Видавництво Національного університету “Львівська політехніка”, 2025. – 25с.

Укладач: асистент кафедри АСУ, д-р філософії Казарян А.Г.

Відповідальний за випуск: Шпак З.Я., к.т.н., доцент

Рецензенти: Березький О.М., д.т.н., професор

Гавриш В.І., д.т.н., професор

Казимира І.Я., к.т.н., доцент

Лабораторна робота №3

Тема: Фронтенд фреймворки

***Мета роботи*:** Отримати практичний досвід роботи з одним із сучасних фронтенд-фреймворків. Навчитись розробляти вебсторінки за допомогою компонентів, керувати станами компонент та працювати з маршрутизацію для переходу між різними відображеннями (сторінками).

Теоретичні відомості

Основи фронтенд-фреймворків

Фронтенд-фреймворк — це структурований набір інструментів, бібліотек і правил, який допомагає розробникам ефективно створювати інтерфейсну частину вебдодатків, тобто ту частину, з якою безпосередньо взаємодіє користувач у браузері. Цей інструмент забезпечує стандартизований підхід до організації коду, компонування елементів, обробки подій та керування станом, що значно полегшує процес розробки, особливо у великих і складних проєктах.

У сучасному веброзробленні інтерфейсні частини вебдодатків стали надзвичайно динамічними, інтерактивними й багатофункціональними. Замість статичних сторінок користувач очікує на швидкий відгук, оновлення вмісту без перезавантаження, персоналізований досвід та приємну візуальну взаємодію. Для досягнення такої якості та гнучкості необхідно використовувати не лише мову JavaScript, а й ефективні засоби її структурування, що дозволяють уникати хаосу у коді та забезпечують повторне використання компонентів. Саме цю функцію виконує фронтенд-фреймворк, упорядковуючи логіку відображення, взаємодії з даними, маршрутизації та інших аспектів, пов’язаних з інтерфейсом.

Роль фронтенд-фреймворку у вебдодатках полягає у забезпеченні уніфікованої моделі для створення складного інтерфейсу, який взаємодіє з користувачем, реагує на його дії, оновлює дані у реальному часі та синхронізується з серверною частиною. Він формує архітектурну основу інтерфейсного коду, дозволяє створювати незалежні елементи (компоненти), кожен з яких відповідає за певну частину логіки або візуального подання. Це дає змогу не тільки спростити підтримку проєкту, а й прискорити командну розробку, де кожен учасник працює над окремими частинами без взаємних конфліктів.

Фронтенд-фреймворк також відіграє важливу роль у розділенні відповідальності між різними частинами вебдодатку. Завдяки чіткому розмежуванню даних, подій і відображення, розробники можуть ізольовано тестувати функціональність, реалізовувати гнучке оновлення інтерфейсу при зміні стану даних, підтримувати масштабованість додатку та уникати

дублювання логіки. Це особливо важливо у сучасних вебсистемах, які мають десятки сторінок, модулів і складну взаємодію з сервером.

Окрім спрощення структури і повторного використання коду, фронтенд-фреймворки забезпечують високу продуктивність додатку. Вони оптимізують оновлення елементів сторінки, мінімізуючи кількість операцій з об'єктною моделлю документа, яка є досить повільною у браузері. Завдяки ефективним алгоритмам оновлення, користувач бачить миттєву реакцію інтерфейсу на свої дії, що створює відчуття швидкості та природної взаємодії.

Фреймворки також спрощують реалізацію таких функцій, як маршрутизація між сторінками, локальне зберігання стану, обробка асинхронних запитів, керування помилками та адаптивне відображення для різних типів пристроїв. Ці можливості вбудовані або легко підключаються у межах фреймворку, що позбавляє розробника потреби реалізовувати всі механізми з нуля.

У контексті вебдодатків фронтенд-фреймворк виступає як платформа, що прискорює розробку, зменшує кількість помилок, забезпечує єдність стилю та поведінки компонентів і, що важливо, дозволяє створювати масштабовані, надійні та зручні у використанні системи. Завдяки високому рівню абстракції, фреймворки усувають потребу працювати з низькорівневими механізмами браузера, дозволяючи зосередитись на логіці взаємодії, дизайні користувацького досвіду та інтеграції з іншими частинами додатку.

Таким чином, фронтенд-фреймворк — це не просто допоміжний інструмент, а необхідний компонент сучасної вебархітектури, що дозволяє розробникам реалізовувати високоякісні рішення у сфері інтерфейсної взаємодії з користувачем. Він формує технологічну базу для швидкої, ефективної та стандартизованої розробки вебдодатків у динамічному цифровому середовищі..

Фреймворки дозволяють розробникам створювати **динамічні користувацькі інтерфейси**, які реагують на зміни в даних, без необхідності ручного оновлення DOM. Вони також пропонують **інструменти та шаблони**, які сприяють **модульній розробці**, полегшуючи створення, підтримку та масштабування вебдодатків. Основною перевагою використання фронтенд-фреймворку є **зменшення кількості повторюваних завдань** і **підвищення продуктивності розробників**, а також забезпечення високої продуктивності роботи користувацького досвіду.

Сучасні вебфреймворки є невід'ємною частиною створення **інтерактивних** і **динамічних** вебсайтів, які зараз широко використовуються в реальних додатках, таких як платформи електронної комерції, соціальні мережі та корпоративні інформаційні панелі.

Використання фронтенд-фреймворків

Розробка за допомогою інтерфейсних фреймворків стала стандартом для створення вебдодатків, оскільки вони спрощують реалізацію багатьох поширених завдань у структурований спосіб. Ось основні причини, чому розробники обирають фронтенд-фреймворки:

- **Архітектура на основі компонентів**: Такі фреймворки, як Angular, React та Vue, дозволяють розробникам створювати додатки з використанням незалежних компонент, які можна використовувати повторно. Такий модульний підхід зменшує дублювання коду і полегшує підтримку та масштабування великих додатків.
- **Оптимізація відображення**: Фронтенд-фреймворки призначені для ефективного відображення змін даних в інтерфейсі. Вони використовують такі технології, як Virtual DOM (React, Vue) та стратегії виявлення змін (Angular), щоб оновлювати лише ті частини сторінки, які потребують змін, замість того, щоб оновлювати всю сторінку.
- **Інструменти та бібліотеки**: Кожен фреймворк має набір інструментів, бібліотек та утиліт, які пришвидшують процес розробки. Сюди входить підтримка процесів збірки, тестування, перевірки якості коду та інтеграція з іншими бібліотеками і внутрішніми сервісами.
- **Екосистема та спільнота**: Фреймворки підтримуються великими, активними спільнотами, що забезпечує регулярні оновлення, виправлення помилок та розробку безліч сторонніх плагінів і розширень.

Ключові риси фронтенд-фреймворків

Фронтенд-фреймворки Angular, React та Vue мають низку спільних концепцій, які дозволяють створювати масштабовані, підтримувані та інтерактивні вебдодатки. Ці риси визначають функціональність, зручність і ефективність використання фреймворку в реальному проєкті.

1. Компонентно-орієнтована архітектура

Усі три фреймворки базуються на ідеї побудови інтерфейсу з незалежних, ізольованих **компонентів**, які мають власну логіку, дані та шаблони. Компоненти можна повторно використовувати, тестувати окремо, комбінувати для створення складних UI.

Приклади:

- React: компоненти створюються як функції або класи з використанням JSX.
- Angular: компоненти реалізуються за допомогою декораторів @Component і шаблонів HTML.
- Vue: компоненти реалізуються як однофайлові .vue файли з блоками <template>, <script> і <style>.

Переваги:

- спрощення структури великого додатку;
- підвищення повторного використання коду;
- чітка ізоляція логіки;
- зручне тестування.

2. Прив'язка даних (Data Binding)

Це механізм синхронізації між **даними в моделі** та **відображенням на екрані (UI)**.

Типи прив’язки:

- Одностороння (one-way): дані передаються від моделі до представлення (наприклад, у React).
- Двостороння (two-way): зміна у UI автоматично оновлює модель, і навпаки (наприклад, у Angular та Vue).

Реалізація:

- Angular: [value]="data" для односторонньої, [(ngModel)]="data" для двосторонньої.
- React: лише одностороння, зміни фіксуються через useState та обробники подій.
- Vue: підтримує v-model для двосторонньої прив’язки.

Користь:

- зменшення потреби в ручному оновленні UI;
- підвищення інтерактивності додатка;
- краща узгодженість даних.

3. Рендеринг на основі змін (Reactivity & DOM Updates)

Фреймворки автоматично оновлюють DOM, коли змінюються дані, без повного перезавантаження сторінки.

Технології:

- React і Vue використовують Virtual DOM — абстрактне представлення реального DOM, яке оптимізує оновлення.
- Angular використовує Change Detection — механізм перевірки змін у стані компонентів.

Переваги:

- висока продуктивність;
- плавна взаємодія з користувачем;
- зменшення навантаження на браузер.

4. Маршрутизація (Routing)

Маршрутизація дозволяє створювати багатосторінкові SPA-додатки (Single Page Applications), де вміст змінюється без перезавантаження всієї сторінки.

Функції маршрутизаторів:

- підтримка URL-адрес;
- передача параметрів;
- захист маршрутів (наприклад, автентифікація);
- вкладені маршрути.

Реалізація:

- Angular: вбудована маршрутизація через RouterModule.
- React: зовнішня бібліотека react-router-dom.
- Vue: офіційна бібліотека vue-router.

5. Керування станом (State Management)

Стан — це дані, які зберігаються в компонентах або централізовано в усьому додатку (глобальний стан). Для складних додатків важливою є централізована система керування станом.

Приклади:

- React: локальний стан через useState, глобальний — через Redux або Context API.
- Vue: локальний — через data(), глобальний — через Vuex (офіційна бібліотека).
- Angular: локальний — через змінні класу, глобальний — через сервіси або сторонні бібліотеки (NGRX).

Користь:

- консистентність даних;
- спрощення обміну інформацією між компонентами;
- масштабування логіки.

1. **Розширення та модульність**

Кожен фреймворк підтримує підключення:

- сторонніх бібліотек (axios, lodash, chart.js);
- розширень (аналітика, автентифікація);
- SSR/SPA рішень (Next.js, Nuxt.js, Angular Universal).

Це дозволяє адаптувати фреймворк до потреб будь-якого проєкту — від блогу до складного ERP-сервісу.

Огляд Angular, React та Vue

Хоча Angular, React і Vue створені з однією метою - створення користувацьких інтерфейсів для сучасних вебдодатків - вони відрізняються за своєю архітектурою, філософією дизайну та екосистемою.

1. Angular (розроблений компанією Google)

Angular - це **повноцінний фронтенд-фреймворк**, який надає цілу екосистему для розробки вебдодатків. Він включає в себе не тільки базові можливості візуалізації інтерфейсу користувача, але й інструменти для роботи з форами, HTTP-запитами та складними сценаріями маршрутизації.

- Найкраще підходить для: Масштабні корпоративні додатки, особливо ті, що потребують повнофункціонального фреймворку з усіма інструментами.

Користь:

- швидка реакція на дії користувача;
- взаємодія з формами, валідацією;
- реалізація анімацій, переходів, меню тощо.

Основні характеристики:

- Використання мови TypeScript: Angular побудовано з використанням TypeScript, тобто розширення мови JavaScript, яка додає статичну типізацію.

- Декларативні шаблони: Angular використовує шаблони на основі HTML, які декларують структуру та поведінку інтерфейсу, що полегшує створення та розуміння макету.
- Директиви: Директиви - це спеціальні атрибути в розмітці HTML, які додають додаткову функціональність елементам (наприклад, ngIf, ngFor).
- Інтеграція залежностей: Angular має потужну систему інтеграції залежностей, яка допомагає ефективніше керувати функціоналом додатку та компонентами.
- Використання в реальному житті:
    - Google Ads, Google Analytics та інші великі сервіси Google.
    - Складні фінансові системи, такі як банківські платформи.
    - Масштабне корпоративне програмне забезпечення.

2. React (розроблений Meta/Facebook)

React - це легка бібліотека JavaScript, розроблена спеціально для створення користувацьких інтерфейсів. На відміну від Angular, вона фокусується в першу чергу на відображенні інтерфейсу і залишає інші аспекти розробки, такі як маршрутизація та управління станами, зовнішнім бібліотекам.

- Найкраще для: Створення інтерактивних, компонентно-керованих користувацьких інтерфейсів для веб та мобільних додатків.
- Основні характеристики:
    - Синтаксис JSX: React використовує JSX, синтаксис, який дозволяє писати HTML-подібний код всередині JavaScript. Даний підхід забезпечує зручний спосіб створення компонентів.
    - Односпрямований потік даних: потоки даних рухаються в одному напрямку, від батьківських компонентів до дочірніх, забезпечуючи передбачуваний стан даних у всьому додатку.
    - Повторне використання компонентів: React наголошує на можливості повторно використовувати компоненти, що покращує підтримку коду та зменшує складність реалізації додатків.
    - Віртуальний DOM: React використовує віртуальний DOM для покращення продуктивності роботи додатків, мінімізуючи кількість оновлень реального DOM.
        - Використання в реальному житті:
- Вебдодатки Facebook, Instagram та WhatsApp.
- Netflix, Airbnb та Uber.
- Складні, інтерактивні платформи електронної комерції.

3. Vue.js (створений спільнотою)

Vue - це прогресивний JavaScript-фреймворк, розроблений для поступової адаптації. На відміну від Angular, Vue менш структурований і дозволяє розробникам використовувати його для створення різних типів вебдодатків - від простих віджетів до повноцінних SPA (односторінкові додатки).

Найкраще підходить для: Швидка розробка малих і середніх додатків або інтеграція в існуючі проекти.

Основні характеристики:

- Декларативний синтаксис: Vue надає синтаксис, який дуже схожий на HTML та CSS, що полегшує роботу розробникам, знайомим з вебстандартами та початок роботи над додатками.
- Реактивне відображення даних: Vue автоматично оновлює інтерфейс користувача, коли змінюється модель даних, що покращує взаємодію з користувачем.
- Vue CLI: Vue надає інструмент командного рядка, який спрощує налаштування проекту та виконання типових завдань розробки.
- Однофайлові компоненти: Vue дозволяє розробникам розробляти компоненти в одному файлі з розділами <template>, <script> та <style>, що спрощує процес розробки.
- Використання в реальному житті:
- Компанії Alibaba, Xiaomi та GitLab.
- Інтерактивні новинні сайти, легкі додатки.
- Внутрішні бізнес-додатки та інформаційні панелі.

Функції, спільні для Angular, React та Vue

Хоча Angular, React і Vue мають свої унікальні особливості, є кілька спільних рис, які роблять ці фреймворки зручними для створення динамічних, сучасних вебдодатків:

1. Компонентно-орієнтована архітектура

Всі три фреймворки підтримують компонентну архітектуру, яка розбиває додатки на менші, багаторазово використовувані одиниці. Компоненти керують власним станом і динамічно відображають інтерфейс користувача. Такий підхід дозволяє розробникам створювати додатки, які легше підтримувати та тестувати, інкапсулюючи логіку та представлення в автономні одиниці.

В усіх трьох фреймворків основна ідея однакова:

- Angular використовує вбудовані декоратори NgModules та Components для реалізації компонентного підходу.
- React використовує функції або класи для створення компонентів.
- Vue використовує однофайлові компоненти (файли .vue).

2. Обробка користувацького вводу та форм

Кожен фреймворк надає своє рішення для управління введенними даними користувачем, перевіркою форм та їх надсиланням на серверну частину. Наприклад:

- Angular надає елементи керування формами, такі як ngModel для двостороннього зв'язування даних та ReactiveForms для більш складного керування формами.
- React керує станом форми через керовані компоненти, використовуючи хуки на кшталт useState.

- Vue підтримує двостороннє зв'язування з v-model для форм, що робить роботу з формами простою та інтуїтивно зрозумілою.

3. Директиви

Кожен фреймворк надає способи додавання користувацької поведінки до HTML-елементів за допомогою спеціальних директив:

- Angular використовує вбудовані директиви, такі як ngIf, ngFor та ngClass.
- React використовує JSX-вирази для додавання логіки у функцію рендерингу, наприклад, умовний рендеринг з { умова ? <Компонент /> : null }.
- Vue використовує директиви v-if, v-for та інші для обробки логіки та рендерингу в шаблонах.

4. Прив'язка даних

Усі три фреймворки працюють з прив'язкою даних, хоча вони відрізняються за способом реалізації. Прив'язка даних гарантує, що зміни даних в одній частині програми автоматично оновлюють відповідні компоненти інтерфейсу.

- Angular підтримує двостороннє зв'язування даних, коли зміни в інтерфейсі поширюються на модель даних, і навпаки.
- React підтримує одностороннє зв'язування даних, коли дані рухаються в одному напрямку (від батьківського до дочірнього компонента), що робить їх більш передбачуваними.
- Vue також підтримує двостороннє зв'язування через v-model для синхронізації інтерфейсу з моделлю даних.

Вибір фреймворку

Вибір між Angular, React та Vue залежить від потреб проекту та досвіду розробників. Angular найкраще підходить для великих додатків корпоративного рівня зі складними вимогами, тоді як React підходить для створення динамічних та інтерактивних інтерфейсів. Vue, будучи простішим і гнучкішим, добре підходить для малих і середніх проектів або проектів, які потребують інтеграції з іншими технологіями. Всі три фреймворки широко використовуються в індустрії, і їх розуміння допоможе вибрати правильний інструмент для проекту з веброзробки.

Порядок виконання роботи

Завдання 1: Створення проекту

Опис: Створити проект за допомогою одного з фреймворків. Всі фреймворки (Angular, React і Vue) мають власні інструменти CLI для швидкого створення нового проекту.

Кроки:

Для Angular:

1. Встановити Angular CLI глобально за допомогою npm:

```
npm install -g @angular/cli
```

Рис.3.1. Встановлення Angular CLI

2. Створити новий Angular-проект:

```
ng new my-angular-app
```

Рис.3.2. Створення нового Angular-проекту

3. Перейти до каталогу проекту і запустити додаток:

```
cd my-angular-app
ng serve
```

Рис.3.3. Перехід до каталогу проекту

За замовчуванням Angular-додаток буде доступний за адресою http://localhost:4200.

Для React:

1. Встановити create-react-app глобально:

```
npm install -g create-react-app
```

Рис.3.4. Встановлення create-react-app глобально

2. Створити новий React-додаток:

```
npx create-react-app my-react-app
```

Рис.3.5. Створення нового React-додатку

3. Перейти до каталогу проекту і запустити сервер розробки:

```
cd my-react-app
npm start
```

Рис.3.6. Запуск сервера розробки

За замовчуванням ваш React-додаток буде доступний за адресою http://localhost:3000.

Для Vue:

1. Встановити Vue CLI глобально за допомогою npm:

```
npm install -g @vue/cli
```

Рис.3.7. Встановлення Vue CLI

2. Створити новий проект Vue:

```
vue create my-vue-app
```

Рис.3.8. Створення проекту Vue

3. Перейти до каталогу проекту і запустіть сервер розробки:

```
cd my-vue-app
npm run serve
```

Рис.3.9. Запуск сервера розробки

За замовчуванням додаток Vue буде доступний за адресою http://localhost:8080.

Завдання 2: Створення компонент

Опис: Реалізація базової компонентної архітектури з використанням обраного фронтенд-фреймворку.

Кроки:

У Angular:

1. Створення компонента: Angular використовує команду ng generate component для створення нового компонента.
2. ng generate < заголовок компонента >
3. Структура компонента: Компонент Angular складається з наступних частин:

- HTML-шаблон: Визначає структуру компонента.
- CSS: Стилізує компонент.
- TypeScript: Містить логіку компонента (наприклад, властивості та методи).
- Декоратор компонента: Декоратор @Component визначає метадані для компонента.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-header',
  templateUrl: './header.component.html',
  styleUrls: ['./header.component.css']
})
export class HeaderComponent {
  title = 'Welcome to My Website';
}
```

Рис.3.10. Приклад компонента Angular

У представленому прикладі заголовок компонента визначається у файлі TypeScript і може бути відображений у шаблоні за допомогою прив'язки даних Angular.

У React:

1. Створення компонента: React використовує JavaScript або JSX для визначення компонентів. Функціональний компонент виглядає так:

```
function Header() {
  return <h1>Welcome to My Website</h1>;
}
```

Рис.3.11. Функціональний компонент

2. Структура компонента React:
   - JSX: HTML-подібний синтаксис, що використовується в JavaScript.
   - Властивості (props): Використовується для передачі даних компонентам.
   - Стан (state): Використовується для керування даними, специфічними для компонента.

```
function Header(props) {
  return <h1>{props.title}</h1>;
}

// Usage
<Header title="Welcome to My Website" />
```

Рис.3.12. Приклад передачі даних через властивості в React

У Vue:

1. Створення компонента: Компоненти Vue зазвичай створюються в одному файлі (файл .vue), який містить шаблон, скрипт і стилі разом.
2. Структура компонента Vue:
   - Шаблон: Визначає структуру HTML.
   - Скрипти (script): Містить логіку JavaScript компонента.
   - Стилі (style): Визначає стилі CSS (можуть бути застосовані до компонента).

```
<template>
  <h1>{{ title }}</h1>
</template>

<script>
export default {
  data() {
    return {
      title: 'Welcome to My Website
    };
  }
};
</script>

<style scoped>
h1 {
  color: blue;
}
</style>
```

Рис.3.13. Приклад базового компонента Vue

Завдання 3: Керування станом у компонентах

Опис: Керування станом всередині компонентів та передача даних між ними за допомогою системи керування станом фреймворку.

Кроки:

У Angular:

Стан компонента: В Angular керування станом здійснюється в межах класу компонента. Є можливість змінювати властивості компонента, після чого шаблон автоматично оновиться, щоб відобразити нові зміни у властивостях.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <div>
      <h1>{{ title }}</h1>
      <button (click)="changeTitle()">Change Title</button>
    </div>
  `,
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title: string = 'Welcome to My Website';

  changeTitle() {
    this.title = 'Title has been changed!';
  }
}
```

Рис.3.14. Приклад керування станом в Angular

У наведеному прикладі властивість title є частиною стану компонента і може бути оновлена за допомогою методу changeTitle().

У React:

Використання стану в React: React використовує функції useState для керування станом у компонентах.

```
import React, { useState } from 'react';

function Header() {
  const [title, setTitle] = useState('Welcome to My Website');

  const changeTitle = () => {
    setTitle('Title has been changed!');
  };

  return (
    <div>
      <h1>{title}</h1>
      <button onClick={changeTitle}>Change Title</button>
    </div>
  );
}
```

Рис.3.15. Використання стану в React

У представленому прикладі useState використовується для визначення стану заголовка, а функція setTitle оновлює його.

In Vue:

Використання стану у Vue: Vue використовує функції data для визначення стану компонента.

```
<template>
  <h1>{{ title }}</h1>
  <button @click="changeTitle">Change Title</button>
</template>

<script>
export default {
  data() {
    return {
      title: 'Welcome to My Website'
    };
  },
  methods: {
    changeTitle() {
      this.title = 'Title has been changed!';
    }
  }
};
</script>
```

Рис.3.16. Використання стану у Vue

У Vue управління станом відбувається всередині функції data(), а зміни автоматично відображаються в шаблоні.

Завдання 4: Реалізація маршрутизації для переходу між сторінками

Опис: Реалізація маршрутизації для навігації між різними представленнями (сторінками) вебдодатку.

Кроки:

1. Реалізувати маршрутизатор у вибраному фреймворку (наприклад, React Router для React, RouterModule для с або Vue Router для Vue).

Для Angular:

- Переконатись, що Angular CLI встановлений і Angular-проект створений.
- Додати Angular Router (попередньо встановлений з Angular):

```
ng add @angular/router
```

Рис.3.17. Додавання Angular Router

Для React:

- Переконатись, що React-проект створений.
- Встановити React Router:

```
npm install react-router-dom
```

Рис.3.18. Встановлення React Router

Для Vue:

- Переконатись, що проект Vue створений.
- Встановити Vue Router:

```
npm install vue-router
```

Рис.3.19. Встановлення Vue Router

2. Створити принаймні два маршрути для вебсайту, наприклад, домашню сторінку та сторінку "Про компанію".

У Angular:

- Відкрити app.routes.ts і визначити маршрути:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component'

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent }
];
```

Рис.3.20. Налаштування маршрутів

- Імпортувати RouterModule та RouterOutlet у кореневий компонент, який використовує (обгортку домашньої сторінки та сторінки About) ці маршрути (наприклад, компонент app.component):

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterModule],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {}
```

Рис.3.21. Імпортувати RouterModule та RouterOutlet

- Додати роутер в шаблон компонента .html, де будуть відображатись дочірні компоненти:

```
<router-outlet></router-outlet>
```

Рис.3.22. Додати роутер в шаблон компонента

- Використати маршрутизатори в компонентах:

```
<a routerLink="/">Home</a>
<a routerLink="/about">About</a>
```

Рис.3.23. Використання маршрутизатора в компонентах

У React:

- Визначити маршрути за допомогою react-router-dom:

```
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom'
import Home from './Home';
import About from './About';

function App() {
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
}

export default App;
```

Рис.3.24. Налаштування маршрутів react-router-dom

In Vue:

- Визначити маршрути в router.js:

```
import { createRouter, createWebHistory } from 'vue-router';
import Home from './views/Home.vue';
import About from './views/About.vue';

const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

export default router;
```

Рис.3.25. Налаштування маршрутів в файлі router.js

- Зареєструвати маршрутизатор як плагін, викликавши функцію use() у додатку в файлі main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from "@/router";

createApp(App).use(router).mount('#app');
```

Рис.3.26. Реєстрація маршрутизатора як плагіна

- Додати роутер в App.vue:

```
<template>
  <nav>
    <router-link to="/">Home</router-link>
    <router-link to="/about">About</router-link>
  </nav>
  <router-view></router-view>
</template>
```

Рис.3.27. Додавання роутера в файл App.vue

3. Додати навігаційні посилання (наприклад, навігаційну панель), щоб користувачі могли перемикатися між сторінками.

4. Переконатись, що коли користувач натискає на посилання, вміст динамічно оновлюється на основі шляху адресної стрічки браузера.

Контрольні запитання

1. Що таке компонентно-орієнтована архітектура у фронтенд-фреймворках, і які її переваги?
2. Які ключові відмінності між Angular, React та Vue у способі керування станом компонента?
3. Що таке Virtual DOM і як він покращує продуктивність у React і Vue?
4. Як працює маршрутизація у фронтенд-фреймворках, і які бібліотеки використовуються для неї у Angular, React та Vue?
5. Що таке директиви у Angular та Vue, і як вони відрізняються від умовного рендерингу у React?
6. Яке призначення фронтенд-фреймворку у сучасному веброзробленні?
7. Які основні переваги використання фронтенд-фреймворків у порівнянні з "чистим" JavaScript?
8. Чому фреймворки важливі для створення динамічного і реагуючого на дії користувача інтерфейсу?
9. Які типові задачі допомагають вирішити фронтенд-фреймворки під час розробки?
10. Як фронтенд-фреймворк сприяє розділенню відповідальності в коді вебдодатка?
11. Що таке компонент у контексті фронтенд-фреймворків і які його ключові властивості?
12. Як компоненти забезпечують модульність та повторне використання коду?
13. Як реалізуються компоненти у Angular, React та Vue?
14. Чим відрізняється структура компонента у Vue (однофайловий .vue-файл) від компонента у React?
15. Які переваги дає ізоляція логіки в межах окремого компонента?
16. Що таке прив’язка даних і які існують її типи?
17. Як реалізується двостороння прив’язка у Angular та Vue?
18. Яким чином React реалізує односторонню прив’язку даних?
19. У чому полягає користь прив’язки даних для взаємодії між інтерфейсом і моделлю?
20. Чому у React не використовується двостороння прив’язка?
21. Що таке Virtual DOM і як він працює у React та Vue?
22. Як Angular визначає необхідність оновлення інтерфейсу користувача?
23. Які переваги має використання Virtual DOM для продуктивності вебдодатка?
24. У чому полягає відмінність між Change Detection у Angular і Virtual DOM у React?
25. Як механізми оновлення DOM впливають на загальний користувацький досвід?
26. Що таке маршрутизація у контексті SPA-додатків?

27. Як реалізується маршрутизація у Angular, і яка роль RouterModule?
28. Яку бібліотеку використовує React для маршрутизації?
29. Як у Vue реалізується підтримка вкладених маршрутів?
30. Чому маршрутизація є важливою складовою сучасних вебдодатків?
31. Що таке локальний і глобальний стан у фронтенд-додатках?
32. Як реалізується керування станом у React за допомогою useState і Context API?
33. Яка роль Vuex у глобальному керуванні станом у Vue?
34. Як Angular використовує сервіси для зберігання і передавання стану між компонентами?
35. Яким чином централізоване керування станом впливає на масштабованість додатків?

Вимоги до оформлення звіту

- Титульна сторінка
- Тема, мета лабораторної роботи
- Результати виконання лабораторної роботи
    - Зображення частини розробленої вебсторінки, що відображає компоненти інтерфейсу користувача створені використовуючи фронт-енд фреймворк.
    - Зображення нової кнопки вебзастосунку доданої відповідно до індивідувального завдання.
    - Зображення сторінок розробленого вебзастосунку, що розташовані за різними URL адресами.

Варіанти завдань

Варіант 1

Вебсайт ресторану

1. Створити компоненти для меню, окремих страв у меню, блоку інгредієнтів страви та відгуків про ресторан.
2. У компоненті Меню додати кнопку для сортування страв у меню за ціною (спадання/зростання) та реалізувати логіку сортування. Відображати компоненти пунктів меню у порядку, що відповідає вибраному типу сортування.
3. Реалізувати маршрутизацію переходів між інформацією про меню, список кухарів та інформацією про ресторан без перезавантаження сторінки в браузері.

Варіант 2

Вебсайт організації онлайн-ігор

1. Створити компоненти для картки гри, картки турніру та блоку статистики гравця.
2. У компоненті "Ігри" додати кнопку для сортування ігор за рейтингом (від найпопулярніших до найменш популярних).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 3

Вебсайт платформи для навчальних курсів

1. Створити компоненти для картки курсу, списку занять у розкладі та блоку прогресу студента.
2. У компоненті "Курси" додати кнопку для сортування курсів за тривалістю (від найкоротших до найдовших). Також, використати групу випадаючих списків (select) для відображення категорій курсів та їх фільтрації за вибраною категорією.
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 4

Вебсайт платформи для бронювання турів

1. Створити компоненти для картки туру, списку заброньованих турів та інтерактивної мапи.
2. У компоненті "Гарячі тури" додати кнопку для сортування турів за ціною (від дешевших до дорожчих).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 5

Вебсайт онлайн-магазину книг

1. Створити компоненти для картки книги, елемента кошика та блоку інформації користувача.

2. У компоненті "Каталог" додати кнопку для фільтрації книг за жанром або автором.

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 6

Вебсайт платформи для оренди автомобілів

1. Створити компоненти для картки автомобіля, списку бронювань та блоку інформації про компанію.

2. У компоненті "Автомобілі" додати кнопку для фільтрації автомобілів за ціною, типом трансмісії та наявністю.

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 7

Вебсайт платформи для оренди житла

1. Створити компоненти для картки квартири, списку забронюваних квартир та інтерактивної мапи.

2. У компоненті "Доступні квартири" додати фільтри для сортування за ціною, кількістю кімнат і типом квартири.

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 8

Вебсайт платформи для онлайн-бронювання квитків на заходи

1. Створити компоненти для картки події, форми бронювання та списку бронювань.

2. У компоненті "Події" додати фільтри для сортування подій за датою та типом заходу (концерти, театри, виставки тощо).

3. Реалізувати маршрутизацію між сторінками подій, моїх бронювань та інформації про організаторів без перезавантаження сторінки.

Варіант 9

Вебсайт платформи для організації волонтерських ініціатив

1. Створити компоненти для картки ініціативи, форми реєстрації на ініціативу та списку приєднаних ініціатив.

2. У компоненті "Доступні ініціативи" додати фільтри для сортування проєктів за датою, місцем та типом активності (екологія, допомога тваринам, соціальна підтримка тощо).

3. Реалізувати маршрутизацію між сторінками ініціатив, моїх ініціатив та інформації про організацію без перезавантаження сторінки.

Варіант 10

Вебсайт онлайн-магазину спортивних товарів

1. Створити компоненти для картки товару, кошика та блоку акцій.

2. У компоненті "Продукти" додати фільтри для сортування товарів за ціною та рейтингом.

3. Реалізувати маршрутизацію між сторінками продуктів, кошика, акцій та профілю без перезавантаження сторінки.

Варіант 11

Вебсайт платформи для пошуку робочих вакансій

1. Створити компоненти для картки вакансії, профілю користувача та блоку пошуку роботи.
2. У компоненті "Вакансії" додати кнопку для сортування вакансій за датою публікації (від найновіших до найстаріших).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 12

Вебсайт платформи для замовлення їжі

1. Створити компоненти для картки страви, кошика та списку замовлень.
2. У компоненті "Меню" реалізувати фільтрацію страв за категоріями (піца, суші, напої тощо).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 13

Вебсайт блогу для подорожей

1. Створити компоненти для картки статті, форми коментарів та списку коментарів.
2. У компоненті "Статті" додати сортування за датою публікації.
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 14

Вебсайт інтерактивної платформи для вивчення мов

1. Створити компоненти для картки уроку, графіку прогресу та тестів.
2. У компоненті "Уроки" реалізувати фільтрацію матеріалів за рівнем складності (A1, A2, B1, B2 тощо).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 15

Вебсайт платформи для вивчення історії через інтерактивні події

1. Створити компоненти для хронології, картки події, тесту та графіку прогресу.
2. У компоненті "Хронологія" реалізувати фільтрацію за періодами (Стародавній світ, Середньовіччя, Новий час тощо).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 16

Вебсайт симулятора управління власним стартапом

1. Створити компоненти для відображення звітів, конкурентів та інвесторів.

2. У компоненті "Ринок" реалізувати фільтрацію конкурентів за сферою діяльності.

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 17

Вебсайт платформи для створення та управління персональними цілями

1. Створити компоненти для картки цілі, прогресу та коментарів у спільноті.

2. У компоненті "Мої цілі" реалізувати фільтрацію за статусом (активні, завершені, відкладені).

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 18

Вебсайт платформи для віртуальних хакатонів та змагань із програмування

1. Створити компоненти для картки хакатону, списку учасників та результатів.

2. У компоненті "Змагання" реалізувати фільтрацію за категоріями (штучний інтелект, веброзробка, кібербезпека тощо).

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 19

Вебсайт симулятора управління будівництвом містом

1. Створити компоненти для картки будівлі, бюджету та індикатора задоволеності жителів.

2. У компоненті "Моє місто" реалізувати фільтрацію доступних об'єктів за категоріями (житлові, комерційні, промислові).

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 20

Вебсайт симулятора космічної експедиції

1. Створити компоненти для корабля, місій та журналу подорожей.

2. У компоненті "Експедиції" реалізувати фільтрацію місій за складністю та типом (дослідницькі, рятувальні, колонізація).

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 21

Вебсайт платформи для моніторингу здоров'я та фізичної активності

1. Створити компоненти для картки тренування, графіку прогресу та плану харчування.

2. У компоненті "Тренування" додати фільтрацію за типами тренувань (кардіо, силові, йога тощо).

3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 22

Вебсайт платформи для онлайн-курсу з фотографії

1. Створити компоненти для картки уроку, галереї та прогресу користувача.
2. У компоненті "Уроки" додати можливість відмічати пройдені уроки.
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 23

Вебсайт платформи для оренди спортивного обладнання

1. Створити компоненти для картки обладнання, списку оренд та форми оплати.
2. У компоненті "Обладнання" реалізувати фільтрацію за типами спорту.
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 24

Вебсайт платформи для планування подорожей

1. Створити компоненти для картки місця для відвідування, списку подорожей та бюджету.
2. У компоненті "Місця для відвідування" реалізувати фільтрацію за типом (пляжі, гірські курорти, міста тощо).
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.

Варіант 25

Вебсайт платформи для створення та обміну рецептами

1. Створити компоненти для картки рецепту, форми для додавання нового рецепту та списку коментарів.
2. У компоненті "Категорії" реалізувати фільтрацію рецептів за типом страви.
3. Реалізувати маршрутизацію між сторінками без перезавантаження сторінки браузера.